

Table of Contents

Handbook	5
Module A.....	5
1. Datatypes in Python:.....	5
1.1 Integer (int)	5
1.2 Float (float)	5
1.3 String (str).....	5
1.4 Boolean (bool)	5
1.5 List (list)	6
1.6 Tuple	6
1.7 Dictionary (dict).....	6
1.8 Set (set).....	6
1.9 None Type (None)	6
1.10 Complex (complex).....	7
2. Logical Operators Overview	7
3. Comparison Operators.....	7
4. Arithmetic Operators in Python	8
Examples of Usage	8
4.2 Advanced Operations	8
4.3 Operations on Python Data Types	8
5. Syntax of if-elif-else Statements.....	10
6. For Loop	10
Syntax:	10
7. While Loop.....	10
Syntax:	10
8. break Statement:	10
Syntax:	10
9. continue Statement:.....	11
Syntax:	11
10. pass Statement:.....	11
Syntax:	11
11. General Functions for Data Types	11
Common Functions	11
12. List Comprehensions	11
Syntax Example.....	11
13. Dictionary Comprehension :.....	12
Example:	12
Explanation:.....	12
14. Lambda Functions:	12
Example:	12
Syntax:	12
15. Recursion :	13
15.1 What is Recursion?.....	13
15.2 How does Recursion work?	13
15.3 Example: Fibonacci Sequence using Recursion	13
16. Variable-Length Arguments (*args) and Keyword Arguments (**kwargs):.....	13
16.1 Keyword Arguments (**kwargs):	13
16.2 Combining *args and **kwargs	13
Example:	14
16.3 Using with Functions:.....	14
17. File Handling in Python	14
Key Concepts	14
17.1 Opening and Closing Files: Use open() to open a file and close() to close it.	14
17.2 Modes:.....	14
17.3 Reading Files: Use read() , readline() , or readlines() to read file content.	14

17.4 Writing Files: Use write() to write data to a file.	14
17.5 Context Manager: Use with to handle files automatically (no need to call close()).....	14
17.6 Opening a File and Reading Content:	14
17.7 Writing to a File:.....	14
17.8 Appending to a File:.....	14
17.9 Reading File Line by Line:	15
17.11 Seek and Tell:.....	15
18. Sorting Techniques and Analysis:	15
18.1 Bubble Sort Concept.....	15
18.2 Quick Sort Concept.....	16
18.3 Merge Sort Concept.....	16
18.4 Insertion Sort Concept:	17
18.5 Selection Sort	18
19. Markov Chain:.....	19
Key Concepts	19
19.1 States:	19
19.2 Transition Probabilities:.....	19
19.3 Initial State Distribution:	19
19.4 Stationary Distribution:.....	19
19.5 Properties	20
19.6 Steps to Solve Markov Chain Problems.....	20
20. NumPy.....	20
20.1 Array.....	20
20.2 Difference between NumPy arrays and lists.....	20
20.3 Higher dimension arrays	21
20.4 Array Properties/Attributes	21
20.5 Accessing Array Elements	21
20.6 Conditional Indexing	21
20.7 Mutability	21
20.8 Referencing	22
20. 9 Reshaping	22
20.10 Data Types Changing data types.....	22
Initialization 20.11 Zeros and Ones.....	22
20.12 Range.....	22
20.13 Operations Arithmetic	23
20.14 Matrix.....	23
Matrix Multiplication	23
20.15 Transpose.....	23
20.16 Determinant.....	24
20.17 Universal Functions	25
21. Random Module	25
21.1 Distribution.....	25
21.2 Seed	25
22. Statistical Analysis	25
22.1 Mean	25
22.2 Median and Mode	26
22.3 Standard deviation and Variance	26
22.4 Visualization with Matplotlib	26
Example:	26
23. Pandas DataFrame.....	26
23.1 Accessing Multiple Columns Concept.....	26
23.2 Introduction to Pandas DataFrames.....	26
23.3 Creating a DataFrame:	27
23.4 Basic Information About the DataFrame:	27
23.5 Accessing Data Accessing Columns:.....	27
Accessing Rows:	27
Summary Statistics:.....	27

Additional Pandas Topics:	28
23.6 Filtering Data:	28
Handling Missing Data:.....	28
23.7 Slicing Rows Concept.....	28
Syntax	28
Example	28
23.8 Accessing First and Last Few Rows Accessing First Rows.....	29
23.9 Syntax	29
Example	29
23.10 Accessing Last Rows Concept.....	29
Syntax	29
Example	29
23.11 Statistical Analysis on Data Concept.....	29
Common Functions	30
Concept.....	30
Syntax	30
Examples.....	30
2. Details of a Specific Student (e.g., Bob):.....	30
3. Specific Columns for a Condition:	31
4. Students with a Specific Age:	31
23.12 Reading Data from CSV Concept	31
Syntax	31
23.13 Getting DataFrame Information Concept.....	31
Finding Specific Values in DataFrame Concept.....	31
Example	32
Module B.....	33
1 Regression: Linear and Polynomial.....	33
1.1 Linear Regression	33
Equation:	33
1.2 Polynomial Regression.....	33
Equation:	33
Properties:	33
1.3 Overfitting vs. Underfitting	33
1.3.1 Overfitting	33
1.3.2 Underfitting	34
2 Principal Component Analysis (PCA).....	34
2.1 Process of PCA.....	34
2.2 Formula.....	34
2.3 Covariance Matrix	34
2.4 Important Properties.....	34
3 PageRank.....	35
3.1 Process	35
3.2 Components and Their Effect	35
4 Neural Networks	35
4.1 Foundations of the Perceptron	35
Prediction:	35
4.2 Enhancing the Perceptron with Learning Algorithms	36
Weight Update Rule:.....	36
4.3 Neuron: Basic Building Block.....	36
4.4 Components of a Neuron	36
4.5 Mathematical Representation	37
4.6 Structure of a Neural Network	37
5 Forward and Backward Propagation	38
5.1 Overview	38
5.2 Forward Pass	38
5.3 Backward Pass.....	39
∂w	40

5.4 Error Calculation.....	40
5.5 Activation Functions	40
6 Convolution neural networks(CNN)	41
6.1 Output Size Calculation	41
Where:.....	41
7 Support Vector Machine (SVM).....	41
7.1 Process	41
7.2 Formula.....	41
7.3 Important Properties.....	41
8 Supervised Learning - Evaluation Metrics	43
8.1 Confusion Matrix	43
8.2 Accuracy	43
8.3 Precision	43
8.4 Recall	43
8.5 Mean Absolute Error (MAE).....	43
n.....	43
8.6 Mean Squared Error (MSE)	43
n.....	43
9 Evaluation Metrics for Unsupervised Learning	45
9.1 1. Silhouette Score	45
Process:	45
Formula:	45
9.2 2. Reconstruction Error (Dimensionality Reduction)	45
Process:	45
Formula:	45
9.3 Important Properties.....	47
Silhouette Score:	47
Reconstruction Error:.....	47
10 K-Means Clustering	47
10.1 Process	47
10.2 Formula.....	47
$ C_j $	47
x_i	47
10.3 Important Properties.....	47

Handbook

Module A

1. Datatypes in Python:

♦ 1.1 Integer (int)

- *Description:* Integers represent whole numbers, both positive and negative, without decimals.
- *Example:*

```
age = 25
temperature = -5
```

- *Explanation:* In this example, age and temperature are integers because they don't have any decimal parts.

♦ 1.2 Float (float)

- *Description:* Floats represent real numbers that have decimal points.
- *Example:*

```
price = 9.99
weight = 70.5
```

- *Explanation:* Both price and weight have decimal parts, which makes them float values.

♦ 1.3 String (str)

- *Description:* Strings are sequences of characters enclosed within quotes. They are used for text.
- *Example:*

```
name = "Alice"
greeting = "Hello, world!"
```

- *Explanation:* Here, name and greeting are strings because they contain characters enclosed in quotes.

♦ 1.4 Boolean (bool)

- *Description:* Booleans represent one of two values: True or False. They are often used in conditional statements.
- *Example:*

```
is_active = True
has_permission = False
```

- *Explanation:* `is_active` and `has_permission` are Boolean values, representing True or False.

- ♦ **1.5 List (list)**

- *Description:* Lists are ordered collections of items, which can be of any data type. Lists are mutable, meaning their contents can be changed.
- *Example:*

```
fruits = ["apple", "banana", "cherry"]  
numbers = [1, 2, 3, 4, 5]
```

- *Explanation:* Here, `fruits` is a list of strings, and `numbers` is a list of integers. Lists are written within square brackets `[]`.

- ♦ **1.6 Tuple**

- *Description:* Tuples are ordered collections of items, similar to lists, but they are immutable, meaning they cannot be changed once created.
- *Example:*

```
coordinates = (10.5, 20.8)  
colors = ("red", "green", "blue")
```

- *Explanation:* `coordinates` and `colors` are tuples. They are enclosed within parentheses `()` and cannot be modified.

- ♦ **1.7 Dictionary (dict)**

- *Description:* Dictionaries are collections of key-value pairs. They are unordered and accessed by unique keys rather than by index.
- *Example:*

```
person = {"name": "Alice", "age": 25, "city": "New York"}
```

- *Explanation:* `person` is a dictionary where each piece of data (like `"name"` and `"age"`) is associated with a unique key. Dictionaries are written within curly braces `{}`.

- ♦ **1.8 Set (set)**

- *Description:* Sets are collections of unique items. They are unordered, meaning there's no guaranteed order for items. Sets automatically eliminate duplicate values.
- *Example:*

```
unique_numbers = {1, 2, 3, 4, 4, 5}
```

- *Explanation:* `unique_numbers` is a set. Even though 4 is added twice, the set stores only unique values `{1, 2, 3, 4, 5}`.

- ♦ **1.9 None Type (None)**

- *Description:* None represents the absence of a value or a null value. It is often used as a placeholder or to indicate "no value."

- *Example:*

```
result = None
```

- *Explanation:* result is set to None, which means it currently has no value.

♦ 1.10 Complex (complex)

- *Description:* Complex numbers consist of a real and an imaginary part, represented as $a + bj$ where a is the real part and b is the imaginary part.

- *Example:*

```
z = 3 + 4j
```

- *Explanation:* z is a complex number where 3 is the real part and $4j$ is the imaginary part.

2. Logical Operators Overview

- ♦ **2.1 and :** Returns True if both conditions are True .

Example:

```
if has_lantern and has_map:  
    print("You are ready!")
```

- ♦ **2.2 or :** Returns True if at least one condition is True . Example:

```
if has_lantern or has_map:  
    print("You can proceed carefully.")
```

- ♦ **2.3 not :** Reverses the Boolean value.

Example:

```
if not has_key:  
    print("You need the key.")
```

3. Comparison Operators

Operator	Description	Example	Output
<	Less than	$5 < 10$	True
>	Greater than	$10 > 5$	True
<=	Less than or equal	$5 <= 5$	True

Operator	Description	Example	Output
>=	Greater than or equal	5 >= 10	False
==	Equal to	5 == 5	True
!=	Not equal to	5 != 10	True

4.Arithmetic Operators in Python

Python provides several operators for performing arithmetic operations.

Operator	Description	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division (float result)	5 / 3	1.666...
//	Floor Division	5 // 3	1
%	Modulus (Remainder)	5 % 3	2
**	Exponentiation	5 ** 3	125

Examples of Usage

♦ 4.1 Basic Arithmetic

```
a = 10
b = 3
print(a + b) # Output: 13
print(a - b) # Output: 7
```

♦ 4.2 Advanced Operations

```
x = 7
y = 2
print(x ** y) # 7 to the power of 2 = 49
print(x // y) # Floor division = 3
print(x % y)  # Remainder = 1
```

♦ 4.3 Operations on Python Data Types

Python provides several operations for different data types. Here's a quick reference:

4.4 Strings

Operation	Description	Example	Result
Indexing	Access a specific character	"hello"[1]	'e'
Slicing	Extract substring	"hello"[1:4]	'ell'
Concatenation	Combine strings	"hello" + " world"	'hello world'
Repetition	Repeat string	"ha" * 3	'hahaha'

4.6 Lists

Operation	Description	Example	Result
Indexing	Access specific element	[1, 2, 3][0]	1
Slicing	Extract sublist	[1, 2, 3, 4][1:3]	[2, 3]
Append	Add element to end	lst = [1]; lst.append(2)	[1, 2]
Extend	Add multiple elements	lst.extend([3, 4])	[1, 2, 3, 4]

4.7 Tuples

Operation	Description	Example	Result
Indexing	Access specific element	(1, 2, 3)[1]	2
Slicing	Extract sub-tuple	(1, 2, 3, 4)[1:3]	(2, 3)
Concatenation	Combine tuples	(1, 2) + (3, 4)	(1, 2, 3, 4)
Repetition	Repeat tuple	(1, 2) * 2	(1, 2, 1, 2)

4.8 Sets

Operation	Description	Example	Result
Add	Add element	s.add(5)	{1, 2, 3, 5}
Remove	Remove element	s.remove(3)	{1, 2}
Union	Combine sets	{1, 2} {3, 4}	{1, 2, 3, 4}
Intersection	Common elements	{1, 2} & {2, 3}	{2}

4.9 Dictionaries

Operation	Description	Example	Result
Access Value	Get value for a key	d["key"]	'value'

Operation	Description	Example	Result
Add/Update Pair	Add or update key-value pair	<code>d["new_key"] = "new_value"</code>	<code>{'key': 'value', 'new_key': 'new_value'}</code>
Keys/Values	Get all keys or values	<code>d.keys() / d.values()</code>	<code>['key'] / ['value']</code>
Delete Pair	Remove a key-value pair	<code>del d["key"]</code>	<code>{}</code>

5.Syntax of if-elif-else Statements

```

if condition1:
    # Code block for condition1
elif condition2:
    # Code block for condition2
else:
    # Code block for when none of the above conditions are true

```

6.For Loop

Used to iterate over sequences like lists, tuples, strings, or ranges.

Syntax:

```

for variable in sequence:
    # Code to execute

```

7. While Loop

Used to repeatedly execute a block of code as long as a condition is True.

Syntax:

```

while condition:
    # Code to execute

```

8.break Statement:

- The `break` statement is used to exit the loop early, regardless of the loop's condition. It terminates the loop immediately and control is transferred to the next statement after the loop.

Syntax:

```
for i in range(10):
    if i == 5:
        break # exit the loop when i is 5
```

9. continue Statement:

- The `continue` statement skips the current iteration of the loop and continues to the next iteration, without executing the remaining code for the current loop iteration.

Syntax:

```
for i in range(10):
    if i == 5:
        continue # skip the rest of the loop when i is 5
    print(i)
```

10. pass Statement:

- The `pass` statement is a placeholder. It is used when no action is required and is usually found in places where code is syntactically required but no action is needed.

Syntax:

```
for i in range(10):
    if i == 5:
        pass # do nothing
    else:
        print(i)
```

11. General Functions for Data Types

Common Functions

- `len()` : Returns the number of items in an object.
- `max()` : Returns the largest item in an object.
- `min()` : Returns the smallest item in an object.
- `sum()` : Returns the sum of items (only for numeric elements).
- `sorted()` : Returns a sorted list of elements from an object.

12. List Comprehensions

List comprehensions provide a concise way to create lists. They combine loops and conditional statements into a single line of code.

Syntax Example

```

numbers = [1, 2, 3, 4, 5]
squares = [x**2 for x in numbers if x > 2]
print(squares) # Output: [9, 16, 25]

```

- **Explanation:** The above code iterates through the list `numbers` , selects elements greater than 2, squares them, and stores the results in the new list `squares` .

13. Dictionary Comprehension :

- A concise way to create dictionaries using loops or conditions within a single line.
- Syntax: {key: value for key, value in iterable if condition} .

Example:

```

numbers = [1, 2, 3, 4, 5]
squares = {num: num**2 for num in numbers if num > 2}
print(squares)

```

Output:

```
{3: 9, 4: 16, 5: 25}
```

Explanation:

- `{num: num**2 for num in numbers if num > 2}` : This dictionary comprehension iterates through the list `numbers` and includes only those numbers greater than 2.
- For each number, it creates a key-value pair where the key is the number (`num`) and the value is the square of the number (`num**2`).

14. Lambda Functions:

- **Purpose:** A small, anonymous function defined with the `lambda` keyword.
- **Syntax:** `lambda arguments: expression`
- Commonly used in short operations, especially with `sorted()` and `map()` .

Example:

```

square = lambda x: x**2
print(square(5)) # Output: 25

```

`sorted()` Function:

- **Purpose:** Sorts elements in an iterable (e.g., a list) based on a key.
- **Syntax:**

```
sorted(iterable, key=function, reverse=False)
```

15. Recursion :

15.1 What is Recursion?

Recursion is a programming technique in which a function calls itself in order to solve smaller instances of the same problem. A recursive function typically has two parts:

1. **Base Case:** The condition under which the function stops calling itself and returns a result. Without a base case, the function would call itself indefinitely.
2. **Recursive Case:** The part where the function calls itself with modified arguments, progressively solving smaller parts of the problem.

15.2 How does Recursion work?

- A recursive function reduces the problem into a simpler or smaller version of the same problem.
- The function keeps calling itself with simpler inputs until it hits the base case.
- The function then starts returning the results and building up the solution step by step.

15.3 Example: Fibonacci Sequence using Recursion

The **Fibonacci sequence** is a series of numbers in which each number is the sum of the two preceding ones. The sequence starts with 0 and 1, and the subsequent numbers are:

- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

The **recursive formula** for Fibonacci is:

- $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ for $n > 1$
- Base cases: $\text{fib}(0) = 0$, $\text{fib}(1) = 1$

16. Variable-Length Arguments (`*args`) and Keyword Arguments (`**kwargs`):

- Allows a function to accept any number of positional arguments.
- Syntax: `*args` collects extra arguments into a tuple.

16.1 Keyword Arguments (`**kwargs`):

- Allows a function to accept any number of keyword arguments.
- Syntax: `**kwargs` collects extra named arguments into a dictionary.

16.2 Combining `*args` and `**kwargs`

- You can use both `*args` and `**kwargs` in the same function to handle both positional and keyword arguments.

```
def describe(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)

describe("artifact1", "artifact2", rarity="rare", year=1985)
# Output:
# Positional arguments: ('artifact1', 'artifact2')
# Keyword arguments: {'rarity': 'rare', 'year': 1985}
```

Example:

16.3 Using with Functions:

- `*args` and `**kwargs` make your functions flexible and reusable by allowing them to accept varying numbers and types of inputs.

17. File Handling in Python

Key Concepts

17.1 Opening and Closing Files: Use `open()` to open a file and `close()` to close it.

17.2 Modes:

- `'r'` : Read (default mode).
- `'w'` : Write (overwrites file if it exists).
- `'a'` : Append (adds content to the end of the file).
- `'r+'` : Read and write.

17.3 Reading Files: Use `read()`, `readline()`, or `readlines()` to read file content.

17.4 Writing Files: Use `write()` to write data to a file.

17.5 Context Manager: Use `with` to handle files automatically (no need to call `close()`).

Examples

17.6 Opening a File and Reading Content:

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content) # Prints the entire file content
```

17.7 Writing to a File:

```
with open("example.txt", "w") as file:
    file.write("Hello, World!")
```

17.8 Appending to a File:

```
with open("example.txt", "a") as file:
    file.write("\nThis is a new line.")
```

17.9 Reading File Line by Line:

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip()) # Removes leading/trailing whitespace
```

Notes:

- Always use with to handle files, as it ensures proper resource management.
- Be cautious with 'w' mode, as it will overwrite existing files.

17.11 Seek and Tell:

Manage the file pointer using seek() and find its position with tell().

```
with open("example.txt", "r") as file:
    print(file.tell()) # Outputs the current file pointer position
    file.seek(5)       # Moves the pointer to the 5th byte
    print(file.read()) # Reads from the 5th byte onwards
```

18. Sorting Techniques and Analysis:

18.1 Bubble Sort Concept

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

Time Complexity

- **Best case:** $O(n)$ – This occurs when the list is already sorted. The algorithm only needs one pass through the list to confirm that no swaps are needed.
- **Worst case:** $O(n^2)$ – This occurs when the list is sorted in reverse order, meaning the algorithm has to compare and swap each element in every pass.

Space Complexity

- $O(1)$ (in-place sorting).

How It Works

1. Compare each pair of adjacent elements.
2. If the first element is larger than the second, swap them.
3. Repeat the process for every element until no swaps are needed.
4. The largest element "bubbles" up to the correct position after each pass.

Example

Initial list: [5, 2, 9, 1, 5, 6]

- After 1st pass: [2, 5, 1, 5, 6, 9]
- After 2nd pass: [2, 1, 5, 5, 6, 9]
- After 3rd pass: [1, 2, 5, 5, 6, 9] (sorted)

18.2 Quick

Sort Concept

Quick Sort is a divide-and-conquer algorithm. It picks a "pivot" element and partitions the list into two sublists: elements smaller than the pivot and elements greater than the pivot. Then, it recursively sorts the sublists.

Time Complexity

- **Best case:** $O(n \log n)$ – This occurs when the pivot divides the list into two nearly equal halves. This results in balanced partitioning at each step, leading to efficient sorting.
- **Worst case:** $O(n^2)$ – This occurs when the pivot is the smallest or largest element in the list, causing the partitioning to be unbalanced (one side has almost all the elements, and the other side has very few), leading to poor performance.

Space Complexity

- $O(\log n)$ (in-place sorting).

How It Works

5. Choose a pivot element from the list (commonly the last element).
6. Partition the list into two sublists: one with elements smaller than the pivot and one with elements greater than the pivot.
7. Recursively apply the same process to the sublists.
8. Once the sublists have been sorted, the entire list is sorted.

Example

Initial list: [10, 80, 30, 90, 40, 50, 70]

- Pivot: 70
- Partitioned: [10, 30, 40, 50, 70, 90, 80]
- Now recursively sort the sublists: [10, 30, 40, 50] and [90, 80]
- **Final sorted list:** [10, 30, 40, 50, 70, 80, 90]

18.3 Merge

Sort Concept

Merge Sort is another divide-and-conquer algorithm. It divides the list into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted list.

Time Complexity

- **Best case:** $O(n \log n)$ – The time complexity is always $O(n \log n)$ because the list is always divided in half, and merging still takes linear time even in the best case.
- **Worst case:** $O(n \log n)$ – Merge Sort performs the same regardless of the initial order of elements because the list is always recursively divided and merged in a consistent manner.

Space Complexity

- $O(n)$ (requires extra space for merging).

How It Works

9. Divide the list into two halves.
10. Recursively sort each half.
11. Merge the sorted halves back together by comparing elements from both halves and adding them to the sorted list.

Example

Initial list: [38, 27, 43, 3, 9, 82, 10]

- Split: [38, 27, 43] , [3, 9, 82, 10]
- Recursively sort: [27, 38, 43] , [3, 9, 10, 82]
- Merge: [3, 9, 10, 27, 38, 43, 82]

18.4 Insertion

Sort Concept:

Insertion Sort is a simple, comparison-based sorting algorithm that builds the final sorted array one item at a time. It is analogous to the way people sort playing cards in their hands.

Time Complexity:

- **Best case:** $O(n)$ – Occurs when the list is already sorted. Only one comparison is needed per element.
- **Worst case:** $O(n^2)$ – Occurs when the list is sorted in reverse order. Each element is compared with all previous elements.

Space Complexity: $O(1)$ (in-place sorting).

How It Works:

12. Divide the list into a "sorted" and "unsorted" section. Initially, the first element is considered sorted.
13. Take the first element from the unsorted section and insert it into its correct position in the sorted section.
14. Repeat the process for all elements in the unsorted section until the entire list is sorted.

Example:

Initial list:

12, 11, 13, 5, 6

1. Start with the second element (11): Insert it into the correct position in

12

. Result:

11, 12, 13, 5, 6

2. Take 13: It is already in the correct position. Result:

11, 12, 13, 5, 6

3. Take 5: Insert it before 11. Result:

5, 11, 12, 13, 6

4. Take 6: Insert it after 5. Result:

5, 6, 11, 12, 13

Final sorted list:

5, 6, 11, 12, 13

18.5 Selection Sort

Concept:

Selection Sort is a comparison-based sorting algorithm that repeatedly selects the smallest (or largest) element from the unsorted section and moves it to the sorted section.

Time Complexity:

- **Best case:** $O(n^2)$ – Even in the best case, the algorithm performs the same number of comparisons as it does in the worst case.
- **Worst case:** $O(n^2)$ – Each element is compared to all others to find the minimum.

Space Complexity: $O(1)$ (in-place sorting).

How It Works:

1. Divide the list into a "sorted" and "unsorted" section. Initially, the entire list is unsorted.
2. Find the smallest element in the unsorted section and swap it with the first element of the unsorted section.
3. Move the boundary between the sorted and unsorted sections one element to the right.
4. Repeat the process until the entire list is sorted.

Example:

Initial list:

64, 25, 12, 22, 11

1. Find the smallest element (11) and swap it with 64. Result:

11, 25, 12, 22, 64

2. Find the next smallest element (12) and swap it with 25. Result:

11, 12, 25, 22, 64

3. Find the next smallest element (22) and swap it with 25. Result:

11, 12, 22, 25, 64

4. 64 is already the largest, no need to swap.

Final sorted list:

11, 12, 22, 25, 64

19. Markov Chain:

A Markov chain is a stochastic process that satisfies the Markov property, which states that the future state of a process depends only on the present state and not on the sequence of events that preceded it. Markov chains are widely used for modeling probabilistic systems that transition between states.

Key Concepts

19.1 States:

- The possible conditions or positions the system can be in.
- Represented as a finite or countable set (e.g., {S1, S2, S3}).

19.2 Transition Probabilities:

- Probabilities associated with moving from one state to another.
- Represented as a matrix (Transition Matrix).
- Example:

$$P = \begin{bmatrix} P(S1 \rightarrow S1), P(S1 \rightarrow S2), P(S1 \rightarrow S3), \\ P(S2 \rightarrow S1), P(S2 \rightarrow S2), P(S2 \rightarrow S3), \\ P(S3 \rightarrow S1), P(S3 \rightarrow S2), P(S3 \rightarrow S3) \end{bmatrix}$$

19.3 Initial State Distribution:

- A vector representing the probabilities of starting in each state.
- Example:

$$\pi = [P(S1 \text{ at } t=0), P(S2 \text{ at } t=0), P(S3 \text{ at } t=0)]$$

19.4 Stationary Distribution:

- A probability distribution that remains unchanged as the system evolves.
- Satisfies:

$$\pi * P = \pi$$

19.5 Properties

15. **Memorylessness:** Future states depend only on the current state.
16. **Irreducibility:** Every state can be reached from every other state.
17. **Aperiodicity:** The system does not oscillate in fixed cycles.
18. **Reversibility:** The Markov chain satisfies detailed balance.

19.6 Steps to Solve Markov Chain Problems

1. **Identify States:** Enumerate all possible states of the system.
2. **Define Transition Matrix:** Construct the matrix of probabilities.
3. **Analyze Transitions:** Compute probabilities for specific sequences or steady-state conditions.
4. **Compute Stationary Distribution:** Solve for π using the stationary condition equation.

20. NumPy

NumPy (Numerical Python) is a foundational library for numerical and scientific computing in Python. It provides robust support for large multi-dimensional arrays and matrices, along with an extensive collection of high-level mathematical functions to manipulate these arrays. NumPy is highly optimized for performance and is a cornerstone in fields like data science, machine learning, and scientific research.

20.1 Array

Basic array creation

An array is a central data structure in NumPy, enabling efficient storage and manipulation of homogeneous data types.

```
import numpy as np

# Creating a 1D array
arr = np.array([1, 2, 3, 4])

# Creating a 2D array
arr_2d = np.array([[1, 2], [3, 4]])

# Creating a 3D array
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

20.2 Difference between NumPy arrays and lists

NumPy arrays offer better performance, more functionality, and memory efficiency compared to Python lists. Unlike lists, NumPy arrays are homogeneous, allowing for optimized operations.

20.3 Higher dimension arrays

Higher-dimensional arrays are useful for applications like image processing or deep learning.

```
# Creating a three-dimensional array
a5 = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4,5,6]]])
print(a5)

# Creating a higher-dimensional array (4D)
high_dim_array = np.ones((2, 3, 4, 5))
```

20.4 Array Properties/Attributes

NumPy arrays have several attributes that provide information about their structure and data.

```
arr = np.array([1, 2, 3, 4])
print(arr.ndim) # Number of dimensions
print(arr.shape) # Shape of the array (dimensions per axis)
print(arr.size) # Total number of elements
print(arr.dtype) # Data type of elements
print(arr.itemsize) # Size of each element in bytes
```

20.5 Accessing Array Elements

Accessing elements in a NumPy array uses zero-based indexing, similar to Python lists.

```
# Accessing elements in a 1D array
print(arr[0])

# Accessing elements in a 2D array
print(arr_2d[0, 1])

# Accessing elements in a 3D array
print(arr_3d[0, 1, 1])
```

20.6 Conditional Indexing

Conditional indexing allows for filtering elements based on a condition.

```
arr = np.array([10,20,30,40,50])
print(arr[arr > 30]) # Retrieves elements greater than 30
arr[arr > 30] = -1 # Updates elements greater than 30 to -1
print(arr)
```

20.7 Mutability

NumPy arrays are mutable, meaning you can modify their contents.

```
arr[0] = 10 # Modifying an element
print(arr)
```

20.8 Referencing

Assigning an array to another variable creates a reference, not a copy. Changes in the new variable affect the original.

```
ref = arr
ref[1] = 20
print(arr) # Original array is also modified
```

20.9 Reshaping

You can reshape arrays to change their structure without altering the data.

```
arr = np.ones((2,3,4), dtype=np.int16)
print("Before reshape:")
print(arr)
arr = arr.reshape(6,4)
print("After reshape:")
print(arr)
```

20.10 Data Types

Changing data types

Some common data types are -

You can explicitly change the data type of array elements using the `astype` method.

```
arr_float = arr.astype(float)
print(arr_float.dtype)
print(arr_float)
```

Initialization

20.11 Zeros and

Ones

You can initialize arrays filled with zeros or ones, useful for default values.

```
zeros = np.zeros((2, 3))
ones = np.ones((2, 3))
arr_eye = np.eye(4) # Generates a square identity matrix
```

20.12 Range

NumPy offers functions to create arrays with numerical ranges.

```
# Creating a range of numbers
arr_range = np.arange(0, 10, 2)

# Creating equally spaced numbers
linspace = np.linspace(0, 1, 5) # Five numbers between 0 and 1
```

20.13 Operations

Arithmetic

Arithmetic operations in NumPy are performed elementwise.

```
sum_result = arr + 2 # Add 2 to each element
mul_result = arr * 3 # Multiply each element by 3
arr_cumsum = np.cumsum(arr) # Finds cumulative sum

arr1 = np.array([10, 11, 12, 13, 14, 15])
arr2 = np.array([20, 21, 22, 23, 24, 25])
print(arr1 + arr2) # Addition
print(arr1 - arr2) # Subtraction
print(arr1 * arr2) # Multiplication
print(arr1 / arr2) # Division
```

20.14 Matrix

Matrix-specific operations are available for 2D arrays.

Matrix Multiplication

```
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
# Element-wise multiplication
m_mul = np.multiply(m1, m2)
# Matrix multiplication
m_dot = np.dot(m1, m2) # Alternatively, use m1 @ m2
```

20.15 Transpose

```
matrix = np.array([[1, 2], [3, 4]])
print(matrix.T) # Transpose of the matrix

matrix = np.array([[1, 2, 3], [4, 5, 6]])
m_transpose = np.transpose(matrix)
```

20.16 Determinant

The determinant of a square matrix is calculated using `numpy.linalg.det`

```
from numpy.linalg
import det
print(det(matrix))
```

20.17 Universal Functions

Universal functions (ufuncs) are vectorized operations applied element-wise.

```
sqrt_result = np.sqrt(arr) # Square root of each element
sin_result = np.sin(arr)   # Sine of each element
```

21. Random Module

NumPy's random module provides functions to generate random numbers.

```
# Generating random numbers between 0 and 1
random_array = np.random.rand(2, 3)

# Generating random integers within a range
random_integers = np.random.randint(0, 10, size=(3, 3))
```

21.1 Distribution

```
numpy.random.rand(d0, d1, ..., dn)
# Uniform Distribution: Generates random numbers to form an array of dimension d
arr = np.random.rand(10000) # This generates 10,000 numbers between 0 and 1

numpy.random.randn(d0, d1, ..., dn)
# Generates numbers from a standard normal distribution
(mean = 0, std = 1)
arr = np.random.randn(10000) # This generates 10,000 numbers from the normal distribution

numpy.random.exponential(scale, size=None)
# Here scale is the inverse of the rate parameter, size determines the number of samples
arr = np.random.exponential(1, 10000)
```

21.2 Seed

Setting a seed ensures that the random numbers generated are reproducible.

```
numpy.random.seed(seed) # seed is an integer value
np.random.seed(182)
print(np.random.rand(10))
```

22. Statistical Analysis

22.1 Mean

The mean (average) is calculated using `numpy.mean`.


```
arr = np.array([1,2,3,4,5])
mean_val = np.mean(arr)
```

22.2 Median and Mode

The median is the middle value, while the mode is the most frequent value.

```
arr = np.array([1,2,3,4,5])
median_val = np.median(arr)
from scipy.stats import mode
mode_val = mode(arr)
```

22.3 Standard deviation and Variance

Standard deviation measures spread, while variance measures the squared spread.

```
arr = np.array([1,2,3,4,5])
std_val = np.std(arr)
var_val = np.var(arr)
```

22.4 Visualization with Matplotlib

NumPy arrays can represent image data, which can be visualized using Matplotlib.

Example:

```
import matplotlib.pyplot as plt
# Generate random image data
image = np.random.random((256, 256, 3))
plt.imshow(image)
plt.show()
```

23. Pandas

DataFrame

23.1 Accessing Multiple Columns Concept

Pandas allows users to access multiple columns by passing a list of column names to the DataFrame. This technique is particularly useful for isolating specific subsets of data for further analysis or visualization.

23.2 Introduction to Pandas DataFrames

- Pandas provides a powerful data structure called **DataFrame**, similar to a table or spreadsheet.
- It allows for easy manipulation and analysis of structured data.

23.3 Creating a DataFrame:

```
import pandas as pd
import numpy as np

student_data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': np.random.randint(18, 22, size=5),
    'Major': ['Computer Science', 'Physics', 'Mathematics', 'Engineering', 'Biology'],
    'GPA': np.random.uniform(2.5, 4.0, size=5).round(2)
}

df = pd.DataFrame(student_data)
print(type(df)) # Output: <class 'pandas.core.frame.DataFrame'>
print(df)
```

23.4 Basic Information About the DataFrame:

```
df.info()
```

- Displays:
 - Number of rows and columns.
 - Data types of each column.
 - Memory usage.

23.5 Accessing Data

Accessing Columns:

```
print(df['Name']) # Access using column name as key
print(df.Age) # Access using dot notation
```

Accessing Rows:

```
print(df.iloc[0]) # Access the first row by index
```

- **iloc**: Access rows by their numerical index.
- **loc**: Access rows by labels (if available).

Summary Statistics:

- Pandas provides functions to compute basic statistics for numerical columns:

```
print(df.describe()) # Summary statistics for numerical columns
```

Additional Pandas Topics:

Adding New Columns:

```
df['Graduation Year'] = 2025
print(df)
```

23.6 Filtering Data:

```
print(df[df['GPA'] > 3.5]) # Students with GPA greater than 3.5
```

Handling Missing Data:

- Fill missing values:

```
df['GPA'].fillna(df['GPA'].mean(), inplace=True)
```

- Drop rows with missing values:

```
df.dropna(inplace=True)
```

23.7 Slicing Rows

Concept

Row slicing allows you to extract specific rows from a DataFrame based on their index positions. This operation is essential for exploring data subsets or preparing training and testing datasets.

Syntax

```
df.iloc[start:stop]
```

- **start**: The starting index (inclusive).
- **stop**: The ending index (exclusive).

Example

```
print(df.iloc[1:4]) # Retrieves rows with index 1, 2, and 3.
```

This operation returns a slice of the DataFrame containing the specified range of rows.

23.8 Accessing First and Last Few Rows

Accessing First Rows

Concept

The `head()` function retrieves the first few rows of the DataFrame. It is useful for previewing the dataset or verifying its structure.

23.9 Syntax

```
df.head(n)
```

- **n**: The number of rows to retrieve (default is 5).

Example

```
print(df.head(2)) # Retrieves the first 2 rows.
```

23.10 Accessing Last Rows

Concept

The `tail()` function retrieves the last few rows of the DataFrame. It is commonly used to check the end of the dataset.

Syntax

```
df.tail(n)
```

- **n**: The number of rows to retrieve (default is 5).

Example

```
print(df.tail(2)) # Retrieves the last 2 rows.
```

23.11 Statistical Analysis on

Data Concept

Pandas offers built-in methods for performing statistical computations and summarizing data. These methods are crucial for understanding data distributions and trends.

Common Functions

1. **describe()**: Provides a statistical summary of numerical columns, including count, mean, standard deviation, minimum, and quartiles.

```
df.describe()
```

2. **mean()**: Calculates the average value of a column.

```
df['column_name'].mean()
```

3. **min()**: Identifies the smallest value in a column.

```
df['column_name'].min()
```

4. **max()**: Identifies the largest value in a column.

```
df['column_name'].max()
```

Concept

Querying enables you to filter rows in a DataFrame based on specific conditions. This technique is pivotal for isolating relevant data.

Syntax

```
df[condition]
```

- **condition**: A logical condition used to filter rows.

Examples

1. **Student with Highest GPA:**

```
df[df['GPA'] == df['GPA'].max()]
```

Retrieves the row where the "GPA" column has the maximum value.

2. **Details of a Specific Student (e.g., Bob):**

```
df[df['Name'] == 'Bob']
```

Filters rows where the "Name" column matches "Bob".

3. Specific Columns for a Condition:

```
df[df['Name'] == 'Bob'][['Major', 'GPA']]
```

Retrieves the "Major" and "GPA" columns for the student named "Bob".

4. Students with a Specific Age:

```
df[df['Age'] == 18]
```

Filters rows where the "Age" column equals 18.

23.12 Reading Data from

CSV Concept

The `read_csv()` function reads data from a CSV file and loads it into a DataFrame. This is a primary method for importing datasets.

Syntax

```
pd.read_csv(filepath)
```

- **filepath**: The file path to the CSV file (string).

23.13 Getting DataFrame

Information Concept

- **info()**: Displays metadata about the DataFrame, such as column names, data types, and non-null values.

```
df.info()
```

This is useful for assessing the dataset's structure and identifying missing data.

- **describe()**: Provides a statistical summary of numerical columns.

```
df.describe()
```

This function is essential for gaining insights into numerical data distributions.

Finding Specific Values in DataFrame

Concept

Conditional indexing enables you to locate rows that meet specific criteria.

Example

1. Day with Maximum Temperature:

```
df[df['Temperature'] == df['Temperature'].max()]
```

Retrieves the row where the "Temperature" column has the maximum value.

Module B

1 Regression: Linear and Polynomial

1.1 Linear Regression

Equation:

$$y = mx + c$$

Properties:

- Assumes a linear relationship between x and y .
- Limited for complex patterns.

1.2 Polynomial Regression

Equation:

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Properties:

- Fits non-linear data by adding polynomial terms.
- Degree n determines model complexity.
- More flexible but prone to overfitting.

1.3 Overfitting vs. Underfitting

1.3.1 Overfitting

Definition: Model captures noise and patterns specific to training data, resulting in poor generalization. **Causes:** High model complexity (e.g., large degree in polynomial regression).

Fixes:

- Reduce model complexity.
- Use regularization (e.g., Ridge or Lasso).
- Increase training data.

1.3.2 Underfitting

Definition: Model is too simple to capture underlying patterns in the data. **Causes:** Low model complexity (e.g., linear regression for non-linear data). **Fixes:**

- Increase model complexity.
- Use higher-degree polynomial terms.
- Improve feature engineering.

2 Principal Component Analysis (PCA)

2.1 Process of PCA

1. Standardize the data to have zero mean and unit variance.
2. Compute the covariance matrix of the data.
3. Calculate the eigenvectors and eigenvalues of the covariance matrix.
4. Select the top k eigenvectors corresponding to the largest k eigenvalues (principal components).
5. Project the data onto the k -dimensional subspace formed by the principal components.

2.2 Formula

$$Z = XW$$

- Z : Transformed data (principal components)
- X : Original data matrix (standardized)
- W : Matrix of eigenvectors (principal components)

2.3 Covariance Matrix

$$C = \frac{1}{n} X^T X$$

- C : Covariance matrix
- X : Standardized data matrix
- n : Number of samples

2.4 Important Properties

- Reduces dimensionality while retaining maximum variance.
- Works by finding orthogonal directions of data variability.
- Sensitive to data scaling; standardization is required.
- Useful for noise reduction and feature extraction.

3 PageRank

3.1 Process

1. **Graph Representation:** Represent the web as a directed graph where nodes are web pages and edges are hyperlinks.
2. **Initial Rank Assignment:** Assign an initial rank to each page (e.g., equally distributed or random).
3. **Rank Update:** Iteratively update ranks based on incoming links and the rank of linking pages.
4. **Convergence:** Continue until the rank values stabilize (i.e., the change is below a small threshold).

3.2 Components and Their Effect

- **Nodes (Web Pages):** Pages to rank.
- **Edges (Hyperlinks):** Connections between pages; more links pointing to a page increase its importance.
- **Outlinks (Outgoing Links):** Pages share their rank among all pages they link to.
- **Damping Factor:** Represents the probability of a random jump to another page, preventing rank concentration.
- **Iterations:** Repeated calculations to redistribute ranks until they converge.

4 Neural Networks

4.1 Foundations of the Perceptron

Prediction:

$$y = \text{sign}(w \cdot x + b)$$

- y : Output label (+1 or -1)
- w : Weight vector
- x : Input feature vector
- b : Bias

4.2 Enhancing the Perceptron with Learning Algorithms

Weight Update Rule:

$$w = w + \Delta w \quad \text{where} \quad \Delta w = \eta(y_{\text{true}} - y_{\text{pred}})x$$

- η : Learning rate
- y_{true} : True label
- y_{pred} : Predicted label

4.3 Neuron: Basic Building Block

A neuron performs two main operations:

1. **Net Operation:** Computes the weighted sum of inputs.
2. **Out Operation:** Applies an activation function to produce the output.

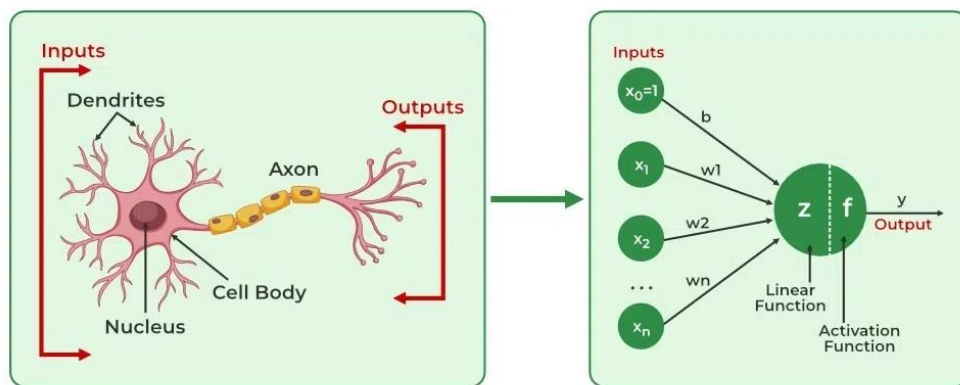


Figure 1: Neuron

4.4 Components of a Neuron

The core elements of a neuron are:

- **Inputs** ($x_1, x_2, \dots, x_n$): Values that represent features or outputs from the previous layer.
- **Weights** ($w_1, w_2, \dots, w_n$): Coefficients that signify the importance of each input.
- **Bias** (b): An additional value that shifts the weighted sum to enhance flexibility.
- **Activation Function** (f): A function that introduces non-linearity to the model, enabling it to learn complex patterns.
- **Output** (y): The final result computed by applying the activation function to the weighted sum of inputs and bias.

4.5 Mathematical Representation

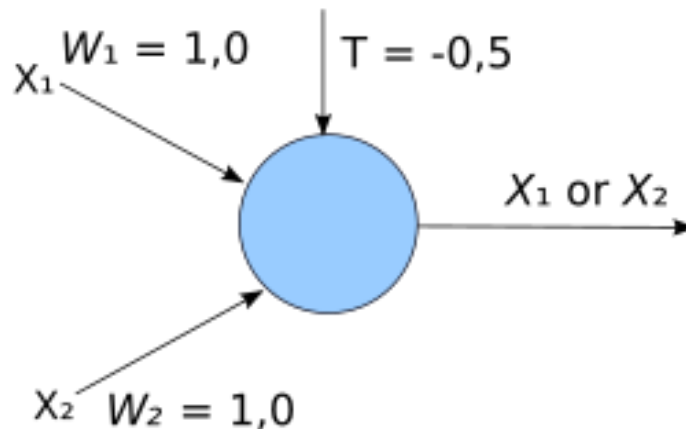


Figure 2: Components of a Neuron

To compute the output of a neuron, first calculate the weighted sum of inputs and bias:

$$\text{Weighted Sum (Net Output)} = \sum_{i=1} w_i x_i + b$$

This value is then passed through the activation function:

$$\text{Output (y)} = f\left(\sum_{i=1}^n w_i x_i + b\right) !$$

In words: The neuron's output is the result of applying the activation function f to the sum of the weighted inputs ($w_i \times x_i$) and the bias (b).

4.6 Structure of a Neural Network

A neural network consists of three main layers:

- **Input Layer:** Accepts raw data and passes it to the next layer.
- **Hidden Layer(s):** Processes inputs using weights and biases, and learns patterns.
- **Output Layer:** Produces the final predictions.

Each neuron performs two steps:

1. Compute the weighted sum of inputs (net operation).
2. Apply the activation function to get the output (out operation).

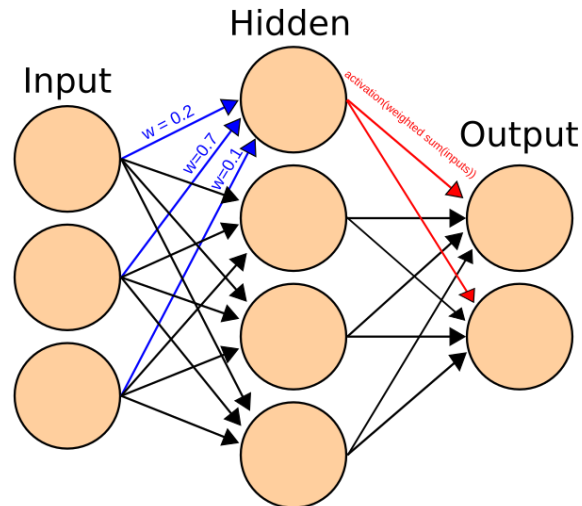


Figure 3: Structure of an Artificial Neural Network

5 Forward and Backward Propagation

5.1 Overview

1. **Forward Pass:** Compute the outputs based on the current weights.
2. **Backward Pass:** Calculate gradients (the rate of change of the error) and propagate errors backward using the chain rule of differentiation.

5.2 Forward Pass

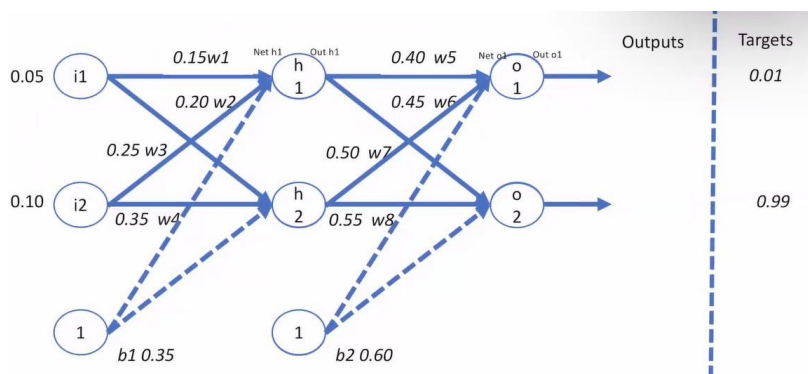


Figure 4: Example of Forward Propagation in a Neural Network

1. **Weighted Sum of Inputs:** For a neuron, the net input is calculated as the weighted sum of the inputs to the neuron, plus a bias term:

$$\text{net}_j = \sum_i w_{ij}x_i + b_j$$

where:

- w_{ij} are the weights associated with the input x_i ,
- b_j is the bias term for neuron j ,
- x_i are the input features.

2. **Activation Function:** The output of the neuron is then obtained by applying an activation function to the net input.

5.3 Backward Pass

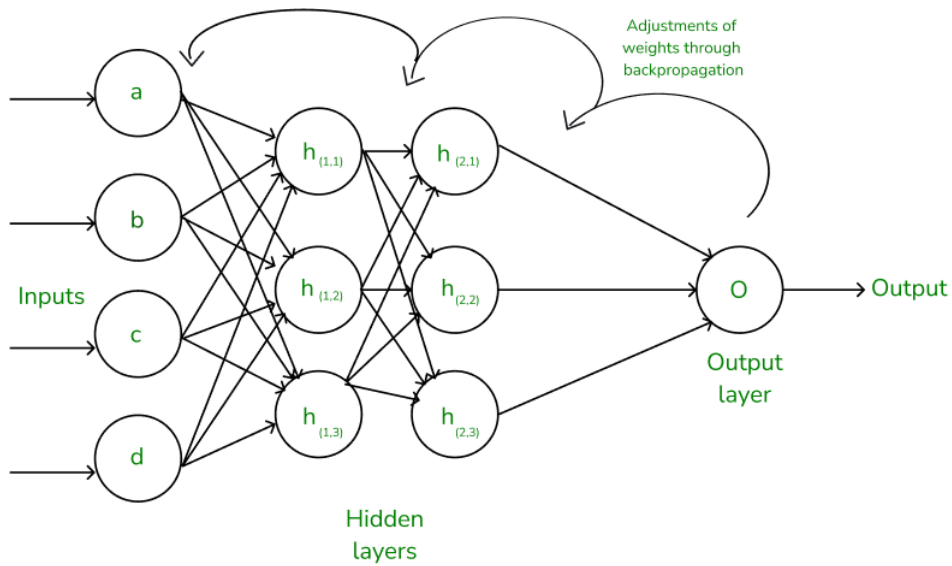


Figure 5: Backward Propagation in an Artificial Neural Network

In the backward pass, we calculate how much each weight in the network contributed to the overall error, and adjust the weights accordingly.

(a) **Error Calculation:** The error for each output neuron is computed using the Mean Squared Error (MSE) formula:

$$E_j = \frac{1}{2}(t_j - y_j)^2$$

where:

- t_j is the target output,
- y_j is the predicted output from the forward pass.

This error measures how far the predicted output y_j is from the desired target output t_j .

(b) **Total Error:** The total error of the network across all output neurons is the sum of the individual errors:

$$E_{\text{total}} = \sum_j E_j$$

- (c) **Gradient Calculation:** To adjust the weights, we need to compute the gradient of the total error with respect to each weight. Using the chain rule of differentiation, the gradient of the total error with respect to a weight w_{ij} is:

$$\frac{\partial E_{\text{total}}}{\partial w_{ij}} = \frac{\partial E_i}{\partial y_j} \cdot \frac{\partial y_j}{\partial \text{net}_j} \cdot \frac{\partial \text{net}_j}{\partial w_{ij}}$$

Each term is computed as follows:

- $\frac{\partial E_i}{\partial y_j} = -(t_j - y_j)$,
- $\frac{\partial y_j}{\partial \text{net}_j} = f'(\text{net}_j)$, where $f'(\text{net}_j)$ is the derivative of the activation function (e.g., for the sigmoid, $f'(\text{net}_j) = y_j(1 - y_j)$),
- $\frac{\partial \text{net}_j}{\partial w_{ij}} = x_i$, the input to the neuron.

- (d) **Weight Update:** The weights are then updated using the gradient descent algorithm:

$$w_{ij}^{\text{new}} = w_{ij} - \eta \cdot \frac{\partial E_{\text{total}}}{\partial w_{ij}}$$

where η is the learning rate, a hyperparameter that controls the step size of the weight update.

5.4 Error Calculation

The total error is calculated using the Mean Squared Error (MSE) function:

$$E_{\text{total}} = \frac{1}{2} \sum_j (t_j - y_j)^2$$

where t_j is the target output and y_j is the predicted output. This error guides the weight update in the backward pass.

5.5 Activation Functions

Activation functions introduce non-linearity into the model. Common types:

- **Sigmoid:** Squashes input into (0, 1). Useful for probabilities.

$$f(x) = \frac{1}{1 + e^{-x}}$$

- **Tanh:** Squashes input into (-1, 1). Centered at zero.

$$f(x) = \tanh(x)$$

- **ReLU:** Replaces negative values with zero.

$$f(x) = \max(0, x)$$

- **Leaky ReLU:** Allows a small gradient for negative values.

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

- **ELU:** Exponential Linear Unit, smoothes ReLU's zero gradient issue.

6 Convolution neural networks(CNN)

6.1 Output Size Calculation

The spatial size of the output feature map in a CNN can be calculated using the formula:

$$O = \frac{W - K + 2P}{S} + 1 \quad (1)$$

Where:

- O : Output feature map size
- W : Input size
- K : Kernel (filter) size
- P : Padding size
- S : Stride

7 Support Vector Machine (SVM)

7.1 Process

- Define Decision Boundary:** Find the hyperplane that best separates classes.
- Maximize Margin:** Ensure the largest distance (margin) between the hyperplane and nearest data points (support vectors).
- Kernel Trick (if required):** Map data to a higher dimension if it is not linearly separable.
- Solve Optimization Problem:** Use quadratic programming to find optimal hyperplane parameters.

7.2 Formula

$$f(x) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

- $\mathbf{w}$: Weight vector, defining the orientation of the hyperplane.
- $\mathbf{x}$: Input vector (data point).
- b : Bias term, shifts the hyperplane.
- $\text{sign}()$: Determines class (+1 or -1).

7.3 Important Properties

- Works well for both linear and non-linear data (with kernel functions).
- Robust to outliers using the soft margin.
- Effective in high-dimensional spaces.
- Relatively memory efficient (uses support vectors only).

8 Supervised Learning - Evaluation Metrics

8.1 Confusion Matrix

A table summarizing the performance of a classification model.

TP	FP
FN	TN

- **TP (True Positive):** Correctly predicted positive cases.
- **FP (False Positive):** Incorrectly predicted as positive.
- **FN (False Negative):** Incorrectly predicted as negative.
- **TN (True Negative):** Correctly predicted negative cases.

—

8.2 Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

8.3 Precision

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

8.4 Recall

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

8.5 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1} |y_i - \hat{y}_i|$$

- y_i : Actual value.
- $\hat{y}_i$: Predicted value.
- n : Total number of samples.

8.6 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1} (y_i - \hat{y}_i)^2$$

- y_i : Actual value.
- $\hat{y}_i$: Predicted value.
- n : Total number of samples.

9 Evaluation Metrics for Unsupervised Learning

9.1 1. Silhouette Score

Process:

- (a) For each data point i , calculate:
 - $a(i)$: Average distance to all other points in the same cluster (intra-cluster distance).
 - $b(i)$: Average distance to all points in the nearest different cluster (inter-cluster distance).
- (b) Compute the silhouette score for each point:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- (c) Calculate the overall silhouette score as the mean of all $s(i)$.

Formula:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $s(i)$: Silhouette score for point i ($-1 \leq s(i) \leq 1$).
- $a(i)$: Average distance to points in the same cluster.
- $b(i)$: Average distance to points in the nearest different cluster.

—

9.2 2. Reconstruction Error (Dimensionality Reduction)

Process:

- (a) Reduce the dimensionality of the data using a method like PCA or autoencoders.
- (b) Reconstruct the data from the reduced dimensions.
- (c) Calculate the error as the difference between the original and reconstructed data.

Formula:

$$\text{Reconstruction Error} = ||X - \hat{X}||^2$$

- X : Original data matrix.
- $\hat{X}$: Reconstructed data matrix from reduced dimensions.
- $|| \cdot ||^2$: Squared norm (e.g., Euclidean distance).

9.3 Important Properties

- **Silhouette Score:**
 - Ranges from -1 to 1 .
 - Positive scores indicate well-separated clusters, while negative scores suggest misclassification.
- **Reconstruction Error:**
 - Lower values indicate better preservation of original data in reduced dimensions.
 - Sensitive to noise and dimensionality reduction techniques.

10 K-Means Clustering

10.1 Process

- (a) **Initialize Centroids:** Randomly select k centroids from the dataset.
- (b) **Assign Clusters:** Assign each data point to the nearest centroid.
- (c) **Update Centroids:** Recalculate the centroids as the mean of points in each cluster.
- (d) **Repeat:** Iterate steps 2 and 3 until centroids stabilize or a maximum number of iterations is reached.

10.2 Formula

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

- μ_j : Centroid of cluster j .
- C_j : Set of points in cluster j .
- x_i : Data point assigned to cluster j .
- $|C_j|$: Number of points in cluster j .

Distance Metric: Commonly uses Euclidean distance:

$$d(x, \mu_j) = \sqrt{\sum_{i=1}^n (x_i - \mu_{j,i})^2}$$

10.3 Important Properties

- **Unsupervised Learning:** No labeled data required.
- **Partitioning Algorithm:** Divides data into k clusters.
- **Centroid-Based:** Minimizes within-cluster variance.

- **Sensitive to Initialization:** Results may vary with random centroids.
- **Scalability:** Efficient for large datasets but may struggle with very high dimensions.